Distributed Computing

Lab Assignment - 9

Name : Naishadh Rana

Enrollment Number : U23CS014

PART 0

1. Environment Variables

```
naish@MacBook-Air-2 Sem6 % docker run --rm -e MY_NAME=Student alpine sh -c 'echo "MY_NAME=$MY_NAME"'
MY_NAME=Student
```

2. Simple shared volume

```
naish@MacBook-Air-2 Sem6 % docker volume create lab9-shared
lab9-shared
naish@MacBook-Air-2 Sem6 % docker run --rm -v lab9-shared:/data alpine sh -c 'echo "hello from container-1" > /data/greeting.txt'
naish@MacBook-Air-2 Sem6 % docker run --rm -v lab9-shared:/data alpine cat /data/greeting.txt
```

I created a named volume and added a file from the first container. Then I started a second container with the same volume mount and read the same file successfully. So, since the data was stored in the volume, not in a single container filesystem, thats why both the containers were able to access the same data.

PART A

3. Docker volumes with redis

```
naish@MacBook-Air-2 Sem6 % docker volume create redis-data
redis-data
naish@MacBook-Air-2 Sem6 % docker run -d --name lab9-redis -v redis-data:/data redis
redis-server --appendonly yes
Unable to find image 'redis:latest' locally
latest: Pulling from library/redis
fbbe1e10a257: Pull complete
b3e1108a590a: Pull complete
577077cdafeb: Pull complete
4f4fb700ef54: Pull complete
0549967a84b5: Pull complete
8015b48063da: Download complete
Digest: sha256:009cc37796fbdbe1b631b4cc0582bed167e5e403ed8bcd06f77eb6cb5aeb6f93
Status: Downloaded newer image for redis:latest
cb70974e7d6ce0c294acb0a0bf28d4d2cd6aa9b84438c882af5402fc71e716cc
naish@MacBook-Air-2 Sem6 % docker exec lab9-redis redis-cli SET course "Distributed
Computing"
docker exec lab9-redis redis-cli GET course
OK
Distributed Computing
naish@MacBook-Air-2 Sem6 % docker rm -f lab9-redis
docker run -d --name lab9-redis -v redis-data:/data redis redis-server --appendonly
yes
lab9-redis
0f2d4a3fd74521bb3fd9931c9e7b443d754197be0d19211ba31fa280368afef8
naish@MacBook-Air-2 Sem6 % docker exec lab9-redis redis-cli GET course
Distributed Computing
```

I ran Redis with a volume mounted and enabled append-only persistence (AOP). Once I added data, even after deleting the container and recreating it, the same volume and data stayed.

Meaning **data persisted across container lifecycle**

4. Kubernetes PVC:

```
naish@MacBook-Air-2 Sem6 % kind create cluster --name lab9
Creating cluster "lab9" ...
 ✓ Ensuring node image (kindest/node:v1.35.0) 🖼
 ✓ Preparing nodes 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
Set kubectl context to "kind-lab9"
You can now use your cluster with:
kubectl cluster-info --context kind-lab9

naish@MacBook-Air-2 Lab9 % kubectl apply -f k8s/manifests/pv-pvc.yaml
persistentvolume/lab9-pv created
persistentvolumeclaim/lab9-pvc created
naish@MacBook-Air-2 Lab9 % kubectl get pv,pvc
NAME                                                       CAPACITY   ACCESS MODES
RECLAIM POLICY   STATUS     CLAIM              STORAGECLASS   VOLUMEATTRIBUTESCLASS
REASON   AGE
persistentvolume/lab9-pv                                   128Mi      RWO
Retain           Bound      default/lab9-pvc   manual         <unset>
0s
persistentvolume/pvc-0c60b8f4-e703-4442-8a3b-15c0a0373f7b   128Mi      RWO
Delete           Released   default/lab9-pvc   standard       <unset>
84s

NAME                              STATUS   VOLUME    CAPACITY   ACCESS MODES
STORAGECLASS   VOLUMEATTRIBUTESCLASS   AGE
persistentvolumeclaim/lab9-pvc   Bound    lab9-pv   128Mi      RWO            manual
<unset>                 0s
naish@MacBook-Air-2 Lab9 % kubectl apply -f k8s/manifests/pvc-pod.yaml
pod/pvc-test-pod created
naish@MacBook-Air-2 Lab9 % kubectl wait --for=condition=Ready pod/pvc-test-pod
--timeout=60s
```

```
pod/pvc-test-pod condition met
naish@MacBook-Air-2 Lab9 % kubectl exec pvc-test-pod -- sh -c 'echo "persistent data"
> /data/test.txt'
naish@MacBook-Air-2 Lab9 % kubectl exec pvc-test-pod -- cat /data/test.txt
persistent data
```

Now deleting the pod and recreating to check if the data persists:

```
naish@MacBook-Air-2 Lab9 % kubectl delete pod pvc-test-pod
pod "pvc-test-pod" deleted from default namespace
naish@MacBook-Air-2 Lab9 % kubectl apply -f k8s/manifests/pvc-pod.yaml
pod/pvc-test-pod created
naish@MacBook-Air-2 Lab9 % kubectl wait --for=condition=Ready pod/pvc-test-pod
--timeout=60s
pod/pvc-test-pod condition met
naish@MacBook-Air-2 Lab9 % kubectl exec pvc-test-pod -- cat /data/test.txt
persistent data
```

- PV and PVC created and bound successfully.
- Pod wrote data to PVC-mounted path.
- Pod deleted and recreated; data survived.

PART B

5. Decoupling Configuration

```
naish@MacBook-Air-2 Lab9 % cd docker/node-id-app
naish@MacBook-Air-2 node-id-app % docker build -t lab9-node-id-app:1.0 .
[+] Building 12.9s (9/9) FINISHED
docker:desktop-linux
 => [internal] load build definition from Dockerfile
0.0s
 => => transferring dockerfile: 126B
0.0s
...
 => => naming to docker.io/library/lab9-node-id-app:1.0
0.0s
 => => unpacking to docker.io/library/lab9-node-id-app:1.0
0.0s

naish@MacBook-Air-2 node-id-app % kind create cluster --name lab9
Creating cluster "lab9" ...
 ✓ Ensuring node image (kindest/node:v1.35.0) 🖼
 ✓ Preparing nodes 📦
 ✓ Writing configuration 📜
 ✓ Starting control-plane 🕹️
 ✓ Installing CNI 🔌
 ✓ Installing StorageClass 💾
Set kubectl context to "kind-lab9"

naish@MacBook-Air-2 node-id-app % kind load docker-image lab9-node-id-app:1.0 --name
lab9
Image: "lab9-node-id-app:1.0" with ID
"sha256:eac10df505c80c3a8a0c2f855f28633ff1a362dd2c8dc1b50477bb5d6acd1405" not yet
present on node "lab9-control-plane", loading...
```

Pod reads NODE_ID:

```
naish@MacBook-Air-2 node-id-app % kubectl wait --for=condition=Ready pod/node-id-pod
--timeout=60s
pod/node-id-pod condition met

naish@MacBook-Air-2 Lab8 % kubectl exec node-id-pod -- python3 -c "import
urllib.request;
print(urllib.request.urlopen('http://localhost:8000').read().decode())"
NODE_ID  : node-alpha
HOSTNAME : node-id-pod
POD_IP   : 10.244.0.5
```

Updating the config map:

```
((.venv) ) naish@MacBook-Air-2 Sem6 % kubectl delete pod node-id-pod
pod "node-id-pod" deleted from default namespace
((.venv) ) naish@MacBook-Air-2 Sem6 % kubectl apply -f k8s/manifests/configmap.yaml
error: the path "k8s/manifests/configmap.yaml" does not exist
((.venv) ) naish@MacBook-Air-2 Sem6 % kubectl apply -f
DC/Labs/Lab9/k8s/manifests/configmap.yaml
configmap/node-config unchanged
naish@MacBook-Air-2 Lab9 % kubectl patch configmap node-config -p
'{"data":{"NODE_ID":"node-beta"}}'
configmap/node-config patched
```

Recreating the pod to see the change:

```
naish@MacBook-Air-2 Lab9 % kubectl apply -f k8s/manifests/configmap-pod.yaml
pod/node-id-pod created
naish@MacBook-Air-2 Lab9 % kubectl wait --for=condition=Ready pod/node-id-pod
--timeout=60s
pod/node-id-pod condition met
naish@MacBook-Air-2 Lab9 % kubectl exec node-id-pod -- wget -qO- http://localhost:8000
NODE_ID  : node-beta
HOSTNAME : node-id-pod
POD_IP   : 10.244.0.7
```

We can see the NODE_ID changed from *node-alpha* to *node-beta* from ConfigMap-injected environment variable.

PART - C

6. Scaling and Service Discovery

Creating 3 Deployment replica and ClusterIP Service

```
naish@MacBook-Air-2 Lab9 % kubectl apply -f k8s/manifests/configmap.yaml
configmap/node-config configured
naish@MacBook-Air-2 Lab9 % kubectl delete pod node-id-pod --ignore-not-found
pod "node-id-pod" deleted from default namespace
naish@MacBook-Air-2 Lab9 % kubectl apply -f k8s/manifests/deployment.yaml
deployment.apps/node-id-deploy created
naish@MacBook-Air-2 Lab9 % kubectl apply -f k8s/manifests/node-id-service.yaml
```

```
naish@MacBook-Air-2 Lab9 % kubectl get pods -l app=node-id -o wide
NAME                              READY   STATUS    RESTARTS   AGE   IP            NODE                 NOMINATED NODE   READINESS GATES
node-id-deploy-66dcd6f775-5wvgq   1/1     Running   0          38m   10.244.0.8    lab9-control-plane   <none>           <none>
node-id-deploy-66dcd6f775-b2xn4   1/1     Running   0          38m   10.244.0.10   lab9-control-plane   <none>           <none>
node-id-deploy-66dcd6f775-t9r6q   1/1     Running   0          38m   10.244.0.9    lab9-control-plane   <none>           <none>
naish@MacBook-Air-2 Lab9 %
```

Sending multiple requests from client to check load balancing:

```
naish@MacBook-Air-2 Lab9 % kubectl apply -f k8s/manifests/client-pod.yaml
pod/client-pod created
naish@MacBook-Air-2 Lab9 % kubectl wait --for=condition=Ready pod/client-pod
--timeout=60s
for i in $(seq 1 6); do
  kubectl exec client-pod -- wget -qO- http://node-id-svc
  echo "---"
done
pod/client-pod condition met
NODE_ID  : node-alpha
HOSTNAME : node-id-deploy-66dcd6f775-5wvgq
POD_IP   : 10.244.0.8
---
NODE_ID  : node-alpha
HOSTNAME : node-id-deploy-66dcd6f775-b2xn4
POD_IP   : 10.244.0.10
---
NODE_ID  : node-alpha
HOSTNAME : node-id-deploy-66dcd6f775-t9r6q
```

```
POD_IP   : 10.244.0.9
---
NODE_ID  : node-alpha
HOSTNAME : node-id-deploy-66dcd6f775-5wvgq
POD_IP   : 10.244.0.8
---
NODE_ID  : node-alpha
HOSTNAME : node-id-deploy-66dcd6f775-t9r6q
POD_IP   : 10.244.0.9
---
NODE_ID  : node-alpha
HOSTNAME : node-id-deploy-66dcd6f775-b2xn4
POD_IP   : 10.244.0.10
---
```

- Deployment with 3 replicas created.
- ClusterIP Service exposed the deployment.
- Multiple requests from client pod showed different HOSTNAME/POD_IP values, demonstrating load balancing.